

Complete AI Dev Pipeline

Assignment #10. Sritej Canchi.

Capstone game: Rex Machina

Concept: A shelter dog walks home across nine days to the family that gave her away, and has to get past the robot dog they replaced her with. Acts 1 and 2 are six timing encounters where something hunts you and telegraphs before it commits. Act 3 is a fifteen round pursuit fight where the robot moves to the tile it predicts you will step on and tells you what it measured off your movement while it does it.

Deliverable 1: playable link

https://sritej.itch.io/rex-machina

Browser build. No download, no login, no instructions needed. Arrow keys or WASD, or the on-screen pad.

Deliverable 2: pipeline source and engine integration

Repository: <https://github.com/SritejCanchi/rex-machina>

Pipeline run video: see submission

Target engine: browser, static HTML5. The build is `index.html`, `game.js` and a folder of JSON.

Automated flow. The pipelines write UE5-style DataTables as JSON. `tools/sync_datatables.py` copies each file into the game's `data/` folder, takes a sha256 of the source and the destination, and refuses to finish if any pair differs. The game then fetches those same files at runtime and reads them directly. There is no transform step, no import script and no hand editing: the bytes the pipeline wrote are the bytes the game parses. `data/MANIFEST.json` records the row count, origin pipeline and hash of every table, and the game prints that summary under the play area, so a player can see that 42 rows across 5 tables came from the pipelines.

Deliverable 3: pipeline audit and cost analysis

1. Pipeline production and functionality

What the pipeline produced, all of it present in the playable build:

Content	Rows	Pipeline
The six Act 1 and 2 chase encounters, each with its telegraph, escape window, fail state and the condition it inflicts	6	A4 Canon Index, RAG
The three Act 3 arenas with exits, cover, lighting and gate moments	3	A4 Canon Index, RAG
Everything the robot says during the fight, 8 read categories across 3 charge registers	24	A4 Canon Index, RAG

Content	Rows	Pipeline
What the robot says when you respawn after losing	6	A6 Retry Read, GER
Journey narration beats	3	A7 Copy Desk, style agent

The game code contains no game content. Every string a player reads, every grid dimension, every exit tile and every escape window is read from `data/`.

What manual steps remain. Three.

1. Uploading the build zip to itch.io is manual.
2. `tools/sync_datatables.py` is run by hand after a pipeline run.
3. The pipelines themselves are invoked by hand rather than on a trigger.

What it would take to eliminate them. One GitHub Action on push: run each pipeline in replay mode, run `sync_datatables.py`, run `node tests/sim.js` and `node qa/adversary.js`, and publish the build with the itch.io Butler CLI if both suites pass. Everything in that chain is already a single command with a non-zero exit code on failure, so it is a workflow file rather than new code.

2. Architectural reflection

What I would change. Recorded model turns are keyed by a sha256 of the entire prompt. Keying on the whole prompt makes replay exact, and it makes a replay provably the answer to the prompt that produced it, which is why I chose it. The cost is that any edit to the prompt invalidates every recording. Changing one sentence in the Copy Desk style guide meant every recorded turn missed, and the pipeline could only run live again. That is most of why the Copy Desk is the most expensive pipeline here: I paid for three full runs to change prose.

The specific alternative. Key the cache on a tuple of the role, the case id, and a digest of the structured inputs only, which is the beat, the word budget, the tone target and the ledger. Store the full prompt alongside the recording as an audit field rather than as the key. A prompt edit then produces a cache hit with a visible warning that the stored prompt differs, which is the information I actually wanted, and it does not force a paid re-run to reword a sentence.

3. Cost analysis

Measured from 150 recorded model turns across the five pipelines, at Claude Haiku 4.5 pricing of \$1 per million input tokens and \$5 per million output. The A6, A7 and A8 figures are the API's own `usage` numbers. A4 and A5 predate usage capture in my recorder, so their tokens are estimated from the stored prompt and response text at four characters per token, and A4's prompts were not stored at all, so its input is missing rather than zero.

Pipeline	Turns	Input	Output	Cost	Share
A7 Copy Desk, style	60	61,154	10,368	\$0.1130	45.7%
A8 Nine Days, narrative	49	28,324	3,780	\$0.0472	19.1%
A4 Canon Index, RAG	15	not stored	9,208 est	\$0.0460	18.6%
A5 Architect	3	6,312 est	4,266 est	\$0.0276	11.2%
A6 Retry Read, GER	23	9,409	804	\$0.0134	5.4%

Pipeline	Turns	Input	Output	Cost	Share
Total	150	105,199	28,426	\$0.2473	

Total actual run cost: \$0.2473. That is every model call made across every pipeline for the whole capstone, not one run.

Most expensive step: the Copy Desk evaluator. It carries 46 percent of the spend on its own. Two reasons. The prompt embeds the full house style guide on every call, roughly 500 tokens, and the loop calls the evaluator once per draft plus once per repair, up to four times per case. Three cases, four calls, three runs is 36 evaluator calls each carrying the whole guide.

Sustainability for a solo developer. Yes, and it is not close. A single playthrough of the shipped game costs nothing, because the content was generated once and ships as static JSON. The \$0.25 above is a one-off content build. Even regenerating everything weekly for a year is about \$13. The cost that would matter is the Nemesis calling the API once per round at runtime, which the GDD budgets at roughly \$0.036 per playthrough. At 10,000 playthroughs a month that is \$360, and that is the number to watch, not the content pipeline.

4. Mid-project cost reduction

Strategy. Every pipeline started as live-only: each run cost API calls, so every debugging pass cost money and every test run cost money. I added a record and replay provider. A live run records each turn keyed by the prompt hash, and every subsequent run replays from disk with no network. Live is now opt-in behind `--live`.

Before: testing the Copy Desk meant 12 to 24 live calls per run. After: `python pipeline.py` reproduces the same output from recorded turns.

Token and cost.

	Before	After
One Copy Desk pipeline run	about 20,000 input, 3,500 output	0
Cost per run	about \$0.037	\$0.00
Reproducible byte for byte by a grader with no API account	no	yes

The second row is the money. The row that mattered more is the third: the evidence files in every submission regenerate from a clean checkout with no key, which turned cost control and reproducibility into the same change.

Rubric self-check

Playable link. Live on itch.io, browser build, no setup.

Pipeline to game connection. 42 rows across 5 tables, each traceable to the pipeline that made it, with hashes in `data/MANIFEST.json` and the count shown in the game.

Engine integration. Verbatim copy with checksum verification. One manual step remains, the upload, named above.

Cost analysis. Measured from 150 real turns, most expensive step identified with the reason, sustainability separated into build cost and runtime cost.

Pipeline audit. Output, three manual steps, one architectural change with a specific alternative, and a before and after on the cost reduction.